ACHTUNG: Diese Aufgabe ist sehr anspruchsvoll und kann wahrscheinlich nur von geübten Programmieren im Rahmen der Zeit bewältigt werden.
Ungeübte Programmierer (jeder möge selbst entscheiden, wie er sich einstuft) sollten von der Vorlage für den finalen Code unter Lab_offbox_final.txt ausgehen, diese analysieren, an die Verhältnisse der Testnetzumgebung anpassen und dann zum Laufen bringen.

1.) Vorgegeben eine Liste mit den Management-Adressen einiger Nexus 9000v Switches und eine Liste einiger CSR1000v Router der Testnetz-Umgebung.
2.) Das Script soll sich per NETCONF mit diesen Systemen verbinden und deren Loopback0-Adressen auslesen und alle in jeweils zwei getrennten Listen (zur weiteren Verwertung) ablegen. Dazu soll das Paket ncclient verwendet werden.
U.U muss auf den Netzelementen NETCONF aktiviert werden:

NX-OS: feature netconf
IOS-XE: netconf-yang
aaa new-model
aaa authentication login default local
aaa authorization exec default local

3.) Der benötigte Subtree Filter für das native IOS-XE YANG Modell

```
filter_ios = '''
   <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <interface>
      <Loopback>
      <name>0</name>
        <ip>
          <address>
            <primary>
              <address/>
              <mask/>
            </primary>
          </address>
        </ip>
      </Loopback>
    </interface>
  </native>
'''
```

4.) Der benötigte Subtree Filter für das native NX-OS YANG Modell

```
filter_nx = '''
   <System xmlns="http://cisco.com/ns/yang/cisco-nx-os-device">
    <ipv4-items>
      <inst-items>
        <dom-items>
          <Dom-list>
          <name>default</name>
            <if-items>
              <If-list>
              <id>lo0</id>
                <addr-items>
                  <Addr-list>
                    <addr/>
                  </Addr-list>
                </addr-items>
              </If-list>
            </if-items>
          </Dom-list>
        </dom-items>
      </inst-items>
    </ipv4-items>
  </System>
'''
```

5.) Die XPath-Expressions zum Parsen des XML Content im RPC Reply:

xpath_ios = './ios:native/ios:interface/ios:Loopback/ios:ip/ios:address/ios:primary/ios:address'

xpath_nx = './nx:System/nx:ipv4-items/nx:inst-items/nx:dom-items/nx:Dom-list/nx:if-items/nx:If-list/nx:addr-items/nx:Addr-list/nx:addr'

Die Pfade sind mit den beteiligten Namespaces qualifiziert:
ns = {'ios':'http://cisco.com/ns/yang/Cisco-IOS-XE-native' , 'nx' : 'http://cisco.com/ns/yang/cisco-nx-os-device'}

6.) Die Namen für den Device Handler: nexus, iosxe
7.) Optional 1: Das Script soll das Skelett einer iBGP-Konfiguration für die Systeme unter ihrer Verantwortung als String mit Umbrüchen generieren unter Vorgabe der ASN (einheitlich 65000). Es soll zu jeder in den vorangegangenen Schritten ermittelten Loopback-Adresse (außer der eigenen) eine iBGP-Nachbarschaft etabliert werden. Als lokale Router ID soll die lokale Loopbback0-Adresse verwendet werden.

8.) Schema der iBGP-Konfiguration im IOS-XE:

```
router bgp <ASN>
bgp router-id <RID>
neighbor <Neighbor IP> remote-as <ASN>
neighbor <Neighbor IP> update-source loopback 0
neighbor <Neighbor IP> next-hop-self
```

9.) Schema der iBGP-Konfiguration im NX-OS:

```
router bgp <ASN>
router-id <RID>
neighbor <Neighbor IP> remote-as <ASN>
     update-source loopback0
     address-family ipv4 unicast
          next-hop-self
```

10.) Tip zur Vorgehensweise: Entfernen die aus den in Schritt 2.) erzeugten Listen die lokale Loopback-Adresse. Erzeugen sie die iBGP-Konfiguration aus zwei Teilen, der globalen Konfiguration und der neighbor-Konfiguration. Letztere wird über die Loopback-Adressen in den Listen iteriert, um damit via Concatenation den gesamten String der BGP-Konfiguration zu erzeugen.

11.) Optional 2: Nutzen Sie netmiko, um die erzeugten Konfigurationen an die Systeme unter ihrer Verantwortung zu pushen. Dazu müssen Sie die String-Vorlage zunächst in eine Liste umwandeln, in der jede Zeile als separater Eintrag erscheint. Nutzen sie dazu die Funktion split() und den Umbruch (\n) als Trennungssymbol. Device Typen für netmiko: cisco_nxos, cisco_ios

HINWEIS: ncclient und netmiko sind auf der Ubuntu VM unter Python 3.7 installiert. Rufen Sie daher ihr Script mit

et@nety-p1:~$ python3.7 <Script Name>.py

auf.